

Dates and Times

Greg Ridgeway Ruth Moyer

2026-07-22

Table of contents

1	Introduction	1
2	New York City Office of Administrative Trials and Hearings (OATH)	3
3	Parsing dates and times	4
4	Extracting parts of a date	9
5	Subtracting dates and <code>difftime()</code>	11
5.1	Estimating time to a decision with censored cases	14
6	Adding time: periods and durations	16
7	Coordinated Universal Time and time zones	18
8	Daylight Saving Time	20
9	Thanksgiving exercises	21
10	Solutions to the exercises	22

1 Introduction

Working with dates and times is very different from working with more familiar numbers. Months have different numbers of days. Some years have 366 days. Clocks restart after 12 or 24 hours. Locations around the world use many different time zones. Most of the United States changes its clocks twice a year for daylight saving time, but places such as Arizona do not. We constantly debate making daylight saving time permanent... or discarding it entirely.

Even apparently simple arithmetic can be ambiguous. Which date is one month after January 31st? Is it February 28th, or should we count 31 days and arrive in March?

Fortunately, software for working with dates exists to make these tasks easier. Unfortunately, systems have made different design decisions. Excel's default date system stores dates as serial numbers, with January 1, 1900 represented by 1. Unix-like systems commonly measure time from January 1, 1970. SPSS measures time in seconds from October 14, 1582, near the introduction of the Gregorian calendar. Much of the world did not adopt that calendar in 1582. Great Britain and its American colonies did not adopt it until 1752. Historians working across calendar systems need to be especially careful.

R has several ways to represent dates and times. We will use the `lubridate` package, which provides readable functions for parsing dates, extracting their components, working with time zones, and doing date arithmetic.

Do not use `as.Date()` or `as.POSIXct()`. `lubridate` has easier-to-read date formatting, handles dates in different formats more intelligently, and has better date arithmetic.

`lubridate` is not part of R by default. You will need to install it. Simply run

```
install.packages("lubridate")
```

and R will hit the web, download the `lubridate` package and any supporting packages it needs, and install them. This is a one-time event. Once you have `lubridate` on your machine you will not need to reinstall it every time you need it.

Some of our students, particularly on Macs, have encountered trouble installing some packages for R. R will sometimes try to download the source code for the packages and compile them from scratch on your machine. Sometimes that goes well and other times it requires that you have other tools installed on your machine. An easy solution is to run

```
install.packages("lubridate", type="mac.binary")
```

instead to tell R to find and install a ready-to-use version of the package.

Once `lubridate` is installed, load it once in each R session where you want to use its features. We will also use `dplyr` to manipulate data.

```
library(lubridate)
library(dplyr)
```

2 New York City Office of Administrative Trials and Hearings (OATH)

This lesson uses records from the New York City Office of Administrative Trials and Hearings (OATH). OATH is an administrative court that hears summonses alleging public-safety and quality-of-life violations. Its [Hearings Division Case Status dataset](#) includes the date and time of a violation, its scheduled hearing, the decision date, and the hearing result.

We will pull data directly from the NYC Open Data API. We will select NYPD-issued summonses with violation dates beginning January 1, 2026. The results may change from one day to the next as OATH posts new records and updates existing cases every day.

The API speaks a query language called Socrata Query Language (SoQL), very similar to SQL. The query below asks for only the columns we need, restricts the records to NYPD summonses from 2026 onward, orders the records by violation date, and allows up to 100,000 results. `URLencode()` converts spaces and punctuation into a form that is safe to include in a web address. Finally, `read.csv()` reads the API's response directly into R.

```
# assemble a Socrata query
oathQuery <-
  "select ticket_number, violation_date, violation_time,
         issuing_agency, charge_1_code_description,
         hearing_date, hearing_time, decision_date, hearing_result
  where violation_date >= '2026-01-01T00:00:00' and
         issuing_agency = 'POLICE DEPARTMENT'
  order by violation_date
  limit 100000" |>
gsub("\n", " ", x=_) |> # remove line breaks
gsub("  +", " ", x=_)   # remove double spaces

oathURL <- paste0(
  "https://data.cityofnewyork.us/resource/jz4z-kudi.csv?$query=",
  URLencode(oathQuery, reserved=TRUE))

dataOATH <- read.csv(oathURL,
                    na.strings=c("", "NA"),
                    colClasses="character")
nrow(dataOATH)
```

```
[1] 27345
```

The query downloaded 27,345 records. If the number of downloaded records reaches the 100,000-row limit, the query will need to be split into pages to ensure that all records are downloaded.

Let's peek at a few rows.

```
dataOATH |>
  slice(c(1, round(n()/2), n()))
```

	ticket_number	violation_date	violation_time	issuing_agency
1	0221377851	2026-01-01T00:00:00.000	16:12:00	POLICE DEPARTMENT
2	0198693092	2026-05-05T00:00:00.000	12:58:00	POLICE DEPARTMENT
3	0220374285	2026-07-16T00:00:00.000	18:15:00	POLICE DEPARTMENT

	charge_1_code_description
1	OPEN CONTAINER CONSUMPTION OF ALCOHOL ON STREETS
2	UNLICENSED GENERAL VENDOR INCLUDING CONT D UNLICENSED ACTIVITY 1ST
3	UNLICENSED GENERAL VENDOR INCLUDING CONT D UNLICENSED ACTIVITY 1ST

	hearing_date	hearing_time	decision_date	hearing_result
1	2026-02-20T00:00:00.000	09:00:00	2026-02-27T00:00:00.000	DEFAULTED
2	2026-06-24T00:00:00.000	09:30:00	2026-06-30T00:00:00.000	DISMISSED
3	2026-09-04T00:00:00.000	09:00:00	<NA>	NONE

Although the values look like dates and times to us, `read.csv()` initially stores them as character strings. R does not yet know that it can perform date arithmetic with them.

```
class(dataOATH$violation_date)
```

```
[1] "character"
```

3 Parsing dates and times

The date portion of `violation_date` is arranged year-month-day, perfect for `lubridate`'s `ymd()` function. Other arrangements have matching functions, including `mdy()` and `dmy()`. When dates are stored as character strings, use year-month-day. It is the only common date format in which ordinary text sorting also produces chronological order. After a string has been converted to a proper `Date` value, R sorts it chronologically regardless of how the date is displayed.

Let's `substring()` the first 10 characters to extract the date part of five scattered values from `violation_date`.

```
textDate <- dataOATH$violation_date[c(1,100,1000,10000,20000)] |>
  substring(1, 10)
textDate
```

```
[1] "2026-01-01" "2026-01-02" "2026-01-12" "2026-04-13" "2026-06-07"
```

```
realDate <- ymd(textDate)
class(realDate)
```

```
[1] "Date"
```

```
realDate
```

```
[1] "2026-01-01" "2026-01-02" "2026-01-12" "2026-04-13" "2026-06-07"
```

It may look like nothing has really changed, but the most important line is the result of `class(realDate)` showing that R knows it is of type `Date` now.

Where `ymd()` becomes really handy is that it can process dates in all kinds of formats. The following are all valid ways to express the same date, and `ymd()` will parse them all correctly.

```
c("2026, July 4",
  "2026, Jul 4",
  "2026-07-04",
  "26-7-4",
  "2026/07/04",
  "2026.07.04",
  "20260704") |>
  ymd()
```

```
[1] "2026-07-04" "2026-07-04" "2026-07-04" "2026-07-04" "2026-07-04"
[6] "2026-07-04" "2026-07-04"
```

You can also use `mdy()` and `dmy()` to parse dates in month-day-year and day-month-year formats, respectively. You need to know which format your data uses. If you choose the wrong function, an ambiguous date may be silently interpreted incorrectly. A date that would require an impossible month or day will instead return `NA`.

```
# incorrectly using ymd() when the month is first
ymd("July 4, 2026")
```

Warning: All formats failed to parse. No formats found.

```
[1] NA
```

```
ymd("7/4/2026")
```

Warning: All formats failed to parse. No formats found.

```
[1] NA
```

```
# incorrectly using dmy()
dmy("7/4/2026") # returns April 7!
```

```
[1] "2026-04-07"
```

```
dmy("7/31/2026") # wrong format, returns NA
```

Warning: All formats failed to parse. No formats found.

```
[1] NA
```

```
# there's a limit
mdy("Jly 7, 2026") # too sloppy
```

Warning: All formats failed to parse. No formats found.

```
[1] NA
```

Note that, when successful, all of these functions produce proper **Date** values, which R displays in year-month-day format. Those **Date** values sort chronologically. If we format them as month-day-year text before sorting, however, R sorts the character strings alphabetically. Do the following seem chronologically sorted to you?

```
format(realDate, "%b %d, %Y") |>
  sort()
```

```
[1] "Apr 13, 2026" "Jan 01, 2026" "Jan 02, 2026" "Jan 12, 2026" "Jun 07, 2026"
```

They are alphabetized instead of chronologically ordered.

The violation date and violation time are in separate columns. `paste()` can join them, and `ymd_hms()` can parse the result as a date and time (year-month-day hours-minutes-seconds). Because all these summonses were issued in New York City, we also tell R that the clock readings use the "America/New_York" time zone.

We will do the same for the hearing date and time and the decision date.

```
dataOATH <- dataOATH |>
  mutate(
    violationDate = ymd_hms(
      paste(substr(violation_date, 1, 10), violation_time),
      tz="America/New_York"),
    hearingDate = ymd_hms(
      paste(substr(hearing_date, 1, 10), hearing_time),
      tz="America/New_York"),
    decisionDate = ymd_hms(
      decision_date, tz="America/New_York"))
```

```
Warning: There was 1 warning in `mutate()`.
i In argument: `hearingDate = ymd_hms(paste(substr(hearing_date, 1, 10),
  hearing_time), tz = "America/New_York")`.
Caused by warning:
! 2 failed to parse.
```

```
dataOATH |>
  select(ticket_number, violationDate,
    hearingDate, decisionDate) |>
  slice(c(1, round(n()/2), n()))
```

	ticket_number	violationDate	hearingDate	decisionDate
1	0221377851	2026-01-01 16:12:00	2026-02-20 09:00:00	2026-02-27
2	0198693092	2026-05-05 12:58:00	2026-06-24 09:30:00	2026-06-30
3	0220374285	2026-07-16 18:15:00	2026-09-04 09:00:00	<NA>

It is worth checking that the new date-time columns are not missing values when the original text columns were not missing. The following three filters should return zero rows.

```
dataOATH |>
  filter(is.na(hearingDate) & !is.na(hearing_date))
```

```
[1] ticket_number      violation_date
[3] violation_time      issuing_agency
[5] charge_1_code_description hearing_date
[7] hearing_time        decision_date
[9] hearing_result      violationDate
[11] hearingDate         decisionDate
<0 rows> (or 0-length row.names)
```

```
dataOATH |>
  filter(is.na(violationDate) & !is.na(violation_date))
```

```
[1] ticket_number      violation_date
[3] violation_time      issuing_agency
[5] charge_1_code_description hearing_date
[7] hearing_time        decision_date
[9] hearing_result      violationDate
[11] hearingDate         decisionDate
<0 rows> (or 0-length row.names)
```

```
dataOATH |>
  filter(is.na(decisionDate) & !is.na(decision_date))
```

```
[1] ticket_number      violation_date
[3] violation_time      issuing_agency
[5] charge_1_code_description hearing_date
[7] hearing_time        decision_date
[9] hearing_result      violationDate
[11] hearingDate         decisionDate
<0 rows> (or 0-length row.names)
```

Let's tidy up by deleting the old date and time columns now that we have properly formatted and stored date objects.

```
dataOATH <- dataOATH |>
  select(-violation_date, -violation_time,
         -hearing_date,   -hearing_time,
         -decision_date)
```

The standardized display is easier for computers to interpret: year, month, day, time, and time-zone abbreviation. More importantly, these columns are now date-time objects rather than text.

```
class(dataOATH$violationDate)
```

```
[1] "POSIXct" "POSIXt"
```

POSIXct is the class R uses for date-time objects. It is a numeric representation of the number of seconds since January 1, 1970, 00:00:00 UTC. The **lubridate** package provides functions to convert between POSIXct and human-readable formats.

4 Extracting parts of a date

Once R recognizes a date, **lubridate** can extract its year, month, day, hour, or weekday. The `label=TRUE` option asks for readable month and weekday labels rather than numbers.

```
exampleDates <- dataOATH |>
  slice(1, 100, 1000, 10000, 20000) |>
  pull(violationDate)

year(exampleDates)
```

```
[1] 2026 2026 2026 2026 2026
```

```
month(exampleDates)
```

```
[1] 1 1 1 4 6
```

```
month(exampleDates, label=TRUE)
```

```
[1] Jan Jan Jan Apr Jun
```

```
12 Levels: Jan < Feb < Mar < Apr < May < Jun < Jul < Aug < Sep < ... < Dec
```

```
month(exampleDates, label=TRUE, abbr=FALSE)
```

```
[1] January January January April June  
12 Levels: January < February < March < April < May < June < ... < December
```

```
wday(exampleDates) # Sunday is 1
```

```
[1] 5 6 2 2 1
```

```
wday(exampleDates, label=TRUE)
```

```
[1] Thu Fri Mon Mon Sun  
Levels: Sun < Mon < Tue < Wed < Thu < Fri < Sat
```

```
wday(exampleDates, label=TRUE, abbr=FALSE)
```

```
[1] Thursday Friday Monday Monday Sunday  
7 Levels: Sunday < Monday < Tuesday < Wednesday < Thursday < ... < Saturday
```

```
hour(exampleDates)
```

```
[1] 16 12 11 20 21
```

Weekday and hour now let us ask when NYPD-issued OATH summonses occur. `week_start=1` starts the labels on Monday rather than Sunday.

```
# count summonses by day of the week  
dataOATH |>  
  count(weekday=wday(violationDate, label=TRUE, week_start=1))
```

	weekday	n
1	Mon	2916
2	Tue	2705
3	Wed	3267
4	Thu	3847
5	Fri	4715
6	Sat	4871
7	Sun	5024

```
# count summonses by hour of the day
dataOATH |>
  count(hour=hour(violationDate)) |>
  arrange(desc(n))
```

	hour	n
1	18	2642
2	19	2559
3	20	2206
4	17	2157
5	21	1913
6	22	1824
7	16	1691
8	23	1523
9	0	1439
10	14	1188
11	13	1140
12	15	1135
13	1	1048
14	12	943
15	11	684
16	2	670
17	3	563
18	10	466
19	4	462
20	9	441
21	8	357
22	5	112
23	7	106
24	6	76

Exercise

1. What are the top five weekday and hour combinations with the most NYPD-issued OATH summonses?

5 Subtracting dates and `difftime()`

When calculating the difference between two dates or times, use `difftime()` instead of the minus sign. The minus sign allows R to choose the units for the result, and R may not

choose the units you actually want. With `difftime()`, you specify the units explicitly, such as `units="days"` or `units="hours"`, which makes both the calculation and your intent clear. Note in the following example that the minus sign returns results with two different time units.

```
# four hour difference
ymd_hm("2026-07-04 16:00") -
  ymd_hm("2026-07-04 12:00")
```

Time difference of 4 hours

```
# one year and four hour difference
ymd_hm("2026-07-04 16:00") -
  ymd_hm("2025-07-04 12:00")
```

Time difference of 365.1667 days

It is dangerous to let R choose units that can change from one calculation to the next. Use `difftime()` and clearly state your desired time units.

Let's use `difftime()` to see how up-to-date NYC OATH data are by comparing the most recent decision date and time in the data with the current date and time.

```
dataOATH |>
  summarize(
    records      = n(),
    firstDecision = min(decisionDate, na.rm=TRUE),
    latestDecision = max(decisionDate, na.rm=TRUE),
    currentTime  = now(tzone="America/New_York"),
    daysBehind   = difftime(currentTime, latestDecision,
                             units="days"))
```

	records	firstDecision	latestDecision	currentTime	daysBehind
1	27345	2026-01-23	2026-07-20	2026-07-22 14:25:24	2.600981 days

We can also calculate the elapsed time between an alleged violation and the currently listed hearing.

```
dataOATH |>
  filter(!is.na(hearingDate)) |>
  mutate(timeToHearing=difftime(hearingDate,
                                violationDate,
                                units="days")) |>
  slice_head(n=5)
```

	ticket_number	issuing_agency		charge_1_code_description	hearing_result
1	0221377851	POLICE DEPARTMENT			
2	0202262720	POLICE DEPARTMENT			
3	0221442587	POLICE DEPARTMENT			
4	0220200577	POLICE DEPARTMENT			
5	0214860333	POLICE DEPARTMENT			

		violationDate	hearingDate	decisionDate	timeToHearing
1		2026-01-01 16:12:00	2026-02-20 09:00:00	2026-02-27	49.70000 days
2		2026-01-01 01:40:00	2026-02-19 09:30:00	2026-02-26	49.32639 days
3		2026-01-01 20:40:00	2026-02-23 09:30:00	2026-03-02	52.53472 days
4	FAILURE TO	2026-01-01 09:25:00	2026-02-20 09:30:00	2026-02-26	50.00347 days
5		2026-01-01 02:40:00	2026-02-20 09:30:00	2026-02-27	50.28472 days

We can create elapsed-time columns and summarize the whole dataset. The decision field is missing for cases that have not yet been resolved. We also exclude negative intervals, which can arise when the portal's current hearing date reflects rescheduling or when a record contains an error.

```
caseTimes <- dataOATH |>
  mutate(daysToHearing = as.numeric(
    difftime(hearingDate, violationDate, units="days")),
    daysToDecision = as.numeric(
    difftime(decisionDate, violationDate, units="days")))

caseTimes |>
  summarize(
    casesWithHearings =
      sum(daysToHearing >= 0, na.rm=TRUE),
```

```

medianDaysToHearing =
  median(daysToHearing[daysToHearing >= 0], na.rm=TRUE),
casesWithDecisions =
  sum(daysToDecision >= 0, na.rm=TRUE),
medianDaysToDecision =
  median(daysToDecision[daysToDecision >= 0], na.rm=TRUE))

```

	casesWithHearings	medianDaysToHearing	casesWithDecisions	medianDaysToDecision
1	27341	50.37639	12897	56.28819

Be careful when interpreting time calculations that drop missing values. Many recent cases have not yet reached a decision. Cases that take longer are less likely to appear among the completed cases, especially near the end of the observation period. As a result, the calculated median may make decisions appear to occur faster than they actually do. This bias comes from ignoring right-censored cases, not from censoring by itself.

5.1 Estimating time to a decision with censored cases

The Kaplan-Meier method lets us include cases that are still awaiting a decision. A completed case contributes the number of days from the violation to the decision. A pending case contributes the number of days from the violation through the date of our analysis. We know that the pending case has lasted at least that long, but we do not yet know its eventual time to a decision. We call that case right censored.

The `Surv()` function combines each case's observed time with an indicator showing whether a decision occurred. The `survfit()` function then estimates the proportion of cases still awaiting a decision as time passes.

We will use the `survival` package to calculate the Kaplan-Meier estimate. This package is included with most standard R installations.

```

library(survival)

analysisDate <- now(tzone="America/New_York")

decisionTimes <- dataOATH |>
  filter(is.na(decisionDate) | decisionDate >= violationDate) |>
  mutate(
    decisionObserved = !is.na(decisionDate) &
      decisionDate <= analysisDate,
    lastObservedDate = if_else(

```

```

      decisionObserved, decisionDate, analysisDate),
      observedDays = as.numeric(
        difftime(lastObservedDate, violationDate, units="days")))) |>
      filter(observedDays >= 0)

decisionFit <- survfit(Surv(observedDays, decisionObserved) ~ 1,
                      data=decisionTimes)

decisionFit

```

Call: `survfit(formula = Surv(observedDays, decisionObserved) ~ 1, data = decisionTimes)`

```

      n events median 0.95LCL 0.95UCL
[1,] 27345  12897   57.2    57.2    57.3

```

The Kaplan-Meier curve begins with all cases awaiting a decision. It steps downward whenever one or more decisions occur. A censored case does not make the curve drop because we did not observe a decision for that case. It still contributes information for every day that it was observed. The estimated median is the point where the curve first reaches 50 percent, meaning that half of cases are estimated to have received decisions and half are still awaiting decisions.

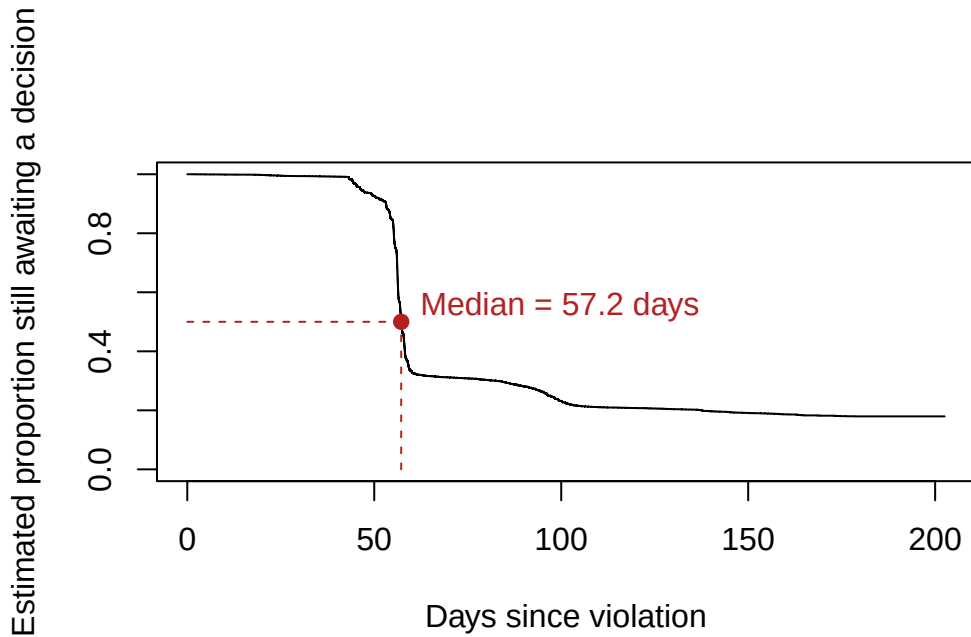
```

medianDecisionDays <- unname(
  summary(decisionFit)$table["median"])

plot(decisionFit,
      conf.int=FALSE,
      mark.time=FALSE,
      xlab="Days since violation",
      ylab="Estimated proportion still awaiting a decision")

if (is.finite(medianDecisionDays))
{
  segments(0, 0.5, medianDecisionDays, 0.5,
           col="firebrick", lty=2)
  segments(medianDecisionDays, 0, medianDecisionDays, 0.5,
           col="firebrick", lty=2)
  points(medianDecisionDays, 0.5,
         col="firebrick", pch=19)
  text(medianDecisionDays, 0.55,
       paste0("Median = ", round(medianDecisionDays, 1), " days"),
       col="firebrick", pos=4)
}

```



Kaplan-Meier assumes that censoring is not informative about the remaining time to a decision. Among otherwise comparable cases that have waited the same number of days, cases censored at that point should have the same prospects for a future decision as cases that continue to be observed. Delayed or selective entry of decisions into the database could violate this assumption.

i Exercises

2. For cases with a nonmissing hearing date, calculate the number of hours from the alleged violation to the scheduled hearing. Store the result in a column named `hoursToHearing` and display the ticket number, violation date, hearing date, and new column for the first five cases.
3. Among cases with nonnegative values of `daysToDecision`, what is the mean number of days from the alleged violation to the decision?

6 Adding time: periods and durations

We can add time to a date. `weeks(1)` represents one calendar week, while `ddays(30)` represents a duration of exactly 30 times 24 hours.

```
exampleViolation <- dataOATH$violationDate[1]
```

```
exampleViolation
```

```
[1] "2026-01-01 16:12:00 EST"
```

```
exampleViolation + weeks(1)
```

```
[1] "2026-01-08 16:12:00 EST"
```

```
exampleViolation + ddays(30)
```

```
[1] "2026-01-31 16:12:00 EST"
```

Calendar time and exact elapsed time are not always the same. In New York, Daylight Saving Time begins on March 8 in 2026. At 2:00 a.m. the clock moves forward to 3:00 a.m., making that local day only 23 hours long.

`days(1)` is a **period**: it advances the calendar by one day and preserves the same local clock time. `ddays(1)` is a **duration**: it adds exactly 24 hours. Notice the different results across the spring clock change.

```
beforeDST <- ymd_hms("2026-03-07 12:00:00", tz="America/New_York")
```

```
beforeDST
```

```
[1] "2026-03-07 12:00:00 EST"
```

```
beforeDST + days(1) # same local time on the next calendar day
```

```
[1] "2026-03-08 12:00:00 EDT"
```

```
beforeDST + ddays(1) # exactly 24 elapsed hours later
```

```
[1] "2026-03-08 13:00:00 EDT"
```

This distinction matters in justice settings. “Return tomorrow at noon” describes a calendar period. “The deadline is exactly 24 hours after filing” describes a duration.

Adding months raises a different problem because months have different lengths. `lubridate` returns a missing value rather than silently inventing a meaning for one month after January 31st.

```
# okay... possible to increase month by 1
ymd("2026-01-01") + months(1)
```

```
[1] "2026-02-01"
```

```
# NA... there is no February 31
ymd("2026-01-31") + months(1)
```

```
[1] NA
```

```
# adds 365.25/12 = 30.4375 days to each date
ymd("2026-01-01") + dmonths(1)
```

```
[1] "2026-01-31 10:30:00 UTC"
```

```
ymd("2026-01-31") + dmonths(1)
```

```
[1] "2026-03-02 10:30:00 UTC"
```

7 Coordinated Universal Time and time zones

Computers need a common reference for representing the same instant around the world. That reference is Coordinated Universal Time, abbreviated UTC. The order of the letters is intentional. According to the [National Institute of Standards and Technology](#), the English name would suggest CUT while the French *temps universel coordonné* would suggest TUC. The International Telecommunication Union selected UTC as a single, language-neutral abbreviation.

The OATH data provide times without an explicit time zone. During our data cleanup, we assigned those dates and times to "America/New_York". Another way to assign a time zone to a date-time that was parsed without the correct zone is `force_tz()`.

```
hearingWithoutCorrectZone <- ymd_hms("2026-07-29 09:30:00")
hearingInNewYork <- force_tz(hearingWithoutCorrectZone,
                             "America/New_York")
```

```
hearingWithoutCorrectZone
```

```
[1] "2026-07-29 09:30:00 UTC"
```

```
hearingInNewYork
```

```
[1] "2026-07-29 09:30:00 EDT"
```

`force_tz()` preserves the numbers on the clock and changes the time zone assigned to them. It is appropriate here because we know that 9:30 was a New York clock reading.

Use `with_tz()` for a different task: displaying the *same instant* as it would appear on a clock somewhere else. For example, someone attending the New York hearing remotely would need to know its local time.

```
hearingInNewYork
```

```
[1] "2026-07-29 09:30:00 EDT"
```

```
with_tz(hearingInNewYork, "America/Los_Angeles")
```

```
[1] "2026-07-29 06:30:00 PDT"
```

```
with_tz(hearingInNewYork, "America/Phoenix")
```

```
[1] "2026-07-29 06:30:00 MST"
```

```
with_tz(hearingInNewYork, "UTC")
```

```
[1] "2026-07-29 13:30:00 UTC"
```

Exercise

4. A remote OATH hearing is scheduled for January 15, 2026, at 9:00 a.m. in New York. Create this date-time in the `America/New_York` time zone, then use `with_tz()` to find the local hearing time in Phoenix.

The `OlsonNames()` function lists the time-zone names R recognizes.

```
head(OlsonNames())
```

```
[1] "Africa/Abidjan"      "Africa/Accra"        "Africa/Addis_Ababa"
[4] "Africa/Algiers"      "Africa/Asmara"       "Africa/Asmera"
```

8 Daylight Saving Time

R's time-zone database knows when Daylight Saving Time begins and ends in each location. `dst()` reports whether a date-time falls in Daylight Saving Time.

```
winter <- ymd_hms("2026-01-15 12:00:00", tz="America/New_York")
summer <- ymd_hms("2026-07-15 12:00:00", tz="America/New_York")

winter
```

```
[1] "2026-01-15 12:00:00 EST"
```

```
dst(winter)
```

```
[1] FALSE
```

```
summer
```

```
[1] "2026-07-15 12:00:00 EDT"
```

```
dst(summer)
```

```
[1] TRUE
```

The autumn change is especially interesting because one hour occurs twice. In New York on November 1, 2026, the clock reaches 1:59 a.m. EDT and then returns to 1:00 a.m. EST. Adding exact one-hour durations makes the repeated hour visible.

```
ymd_hms("2026-11-01 00:30:00", tz="America/New_York") + dhours(0:3)
```

```
[1] "2026-11-01 00:30:00 EDT" "2026-11-01 01:30:00 EDT"
[3] "2026-11-01 01:30:00 EST" "2026-11-01 02:30:00 EST"
```

This is why a timestamp such as “November 1 at 1:30 a.m.” is incomplete: there are two such instants that morning. The date, clock time, and time zone (or UTC offset) are needed to identify the time unambiguously.

We can have R identify the transition by generating minutes around it and grouping by DST status.

```
data.frame(date=ymd_hms("2026-11-01 00:00:00",
                        tz="America/New_York") +
            dminutes(0:240)) |>
  group_by(inDaylightSavingTime=dst(date)) |>
  summarize(first=min(date), last=max(date))
```

```
# A tibble: 2 x 3
  inDaylightSavingTime first          last
<lgl>                <dtm>          <dtm>
1 FALSE              2026-11-01 01:00:00 2026-11-01 03:00:00
2 TRUE               2026-11-01 00:00:00 2026-11-01 01:59:00
```

9 Thanksgiving exercises

Exercises

5. Thanksgiving occurs on the fourth Thursday in November. On what date will Thanksgiving fall in 2026? List all dates in November, use `wday()` to identify Thursdays, then select the fourth Thursday.
6. Make a function that takes a year as input and returns the date of Thanksgiving in that year.

Here is a template for the function.

```

tday <- function(year)
{
  return( )
}

```

10 Solutions to the exercises

1. What are the top five weekday and hour combinations with the most NYPD-issued OATH summonses?

```

dataOATH |>
  count(weekday=wday(violationDate, label=TRUE, week_start=1),
        hour=hour(violationDate)) |>
  arrange(desc(n)) |>
  slice_head(n=5)

```

	weekday	hour	n
1	Sun	18	548
2	Sun	19	545
3	Fri	18	542
4	Fri	22	491
5	Fri	21	486

2. For cases with a nonmissing hearing date, calculate the number of hours from the alleged violation to the scheduled hearing. Store the result in a column named `hoursToHearing` and display the ticket number, violation date, hearing date, and new column for the first five cases.

```

dataOATH |>
  filter(!is.na(hearingDate)) |>
  mutate(hoursToHearing=difftime(hearingDate,
                                violationDate,
                                units="hours")) |>
  select(ticket_number, violationDate, hearingDate,
        hoursToHearing) |>
  slice_head(n=5)

```

	ticket_number	violationDate	hearingDate	hoursToHearing
1	0221377851	2026-01-01 16:12:00	2026-02-20 09:00:00	1192.800 hours
2	0202262720	2026-01-01 01:40:00	2026-02-19 09:30:00	1183.833 hours
3	0221442587	2026-01-01 20:40:00	2026-02-23 09:30:00	1260.833 hours
4	0220200577	2026-01-01 09:25:00	2026-02-20 09:30:00	1200.083 hours
5	0214860333	2026-01-01 02:40:00	2026-02-20 09:30:00	1206.833 hours

3. Among cases with nonnegative values of `daysToDecision`, what is the mean number of days from the alleged violation to the decision?

```
caseTimes |>
  filter(daysToDecision >= 0) |>
  summarize(meanDaysToDecision=mean(daysToDecision, na.rm=TRUE))
```

```
meanDaysToDecision
1          59.0098
```

4. A remote OATH hearing is scheduled for January 15, 2026, at 9:00 a.m. in New York. Create this date-time in the `America/New_York` time zone, then use `with_tz()` to find the local hearing time in Phoenix.

```
ymd_hm("2026-01-15 09:00", tz="America/New_York") |>
  with_tz("America/Phoenix")
```

```
[1] "2026-01-15 07:00:00 MST"
```

5. Thanksgiving occurs on the fourth Thursday in November. On what date will Thanksgiving fall in 2026? List all dates in November, use `wday()` to identify Thursdays, then select the fourth Thursday.

```
data.frame(date=mdy("11/1/2026") + ddays(0:29)) |>
  filter(wday(date, label=TRUE) == "Thu") |>
  slice(4)
```

```
date
1 2026-11-26
```

```
# or using some base R code
a <- mdy(paste0("11/",1:30,"/2026"))
a[wday(a,label=TRUE)=="Thu"][4]
```

```
[1] "2026-11-26"
```

6. Make a function that takes a year as input and returns the date of Thanksgiving in that year.

```
tday <- function(year)
{
  data.frame(date=mdy(paste0("11/1/",year)) + ddays(0:29)) |>
    filter(wday(date, label=TRUE) == "Thu") |>
    slice(4) |>
    pull(date)
}
tday(2026)
```

```
[1] "2026-11-26"
```

```
sapply(2026:2100, tday) |> as_date() # sapply() strips the Date class
```

```
[1] "2026-11-26" "2027-11-25" "2028-11-23" "2029-11-22" "2030-11-28"
[6] "2031-11-27" "2032-11-25" "2033-11-24" "2034-11-23" "2035-11-22"
[11] "2036-11-27" "2037-11-26" "2038-11-25" "2039-11-24" "2040-11-22"
[16] "2041-11-28" "2042-11-27" "2043-11-26" "2044-11-24" "2045-11-23"
[21] "2046-11-22" "2047-11-28" "2048-11-26" "2049-11-25" "2050-11-24"
[26] "2051-11-23" "2052-11-28" "2053-11-27" "2054-11-26" "2055-11-25"
[31] "2056-11-23" "2057-11-22" "2058-11-28" "2059-11-27" "2060-11-25"
[36] "2061-11-24" "2062-11-23" "2063-11-22" "2064-11-27" "2065-11-26"
[41] "2066-11-25" "2067-11-24" "2068-11-22" "2069-11-28" "2070-11-27"
[46] "2071-11-26" "2072-11-24" "2073-11-23" "2074-11-22" "2075-11-28"
[51] "2076-11-26" "2077-11-25" "2078-11-24" "2079-11-23" "2080-11-28"
[56] "2081-11-27" "2082-11-26" "2083-11-25" "2084-11-23" "2085-11-22"
[61] "2086-11-28" "2087-11-27" "2088-11-25" "2089-11-24" "2090-11-23"
[66] "2091-11-22" "2092-11-27" "2093-11-26" "2094-11-25" "2095-11-24"
[71] "2096-11-22" "2097-11-28" "2098-11-27" "2099-11-26" "2100-11-25"
```